



PARIS PANTHÉON-ASSAS UNIVERSITÉ

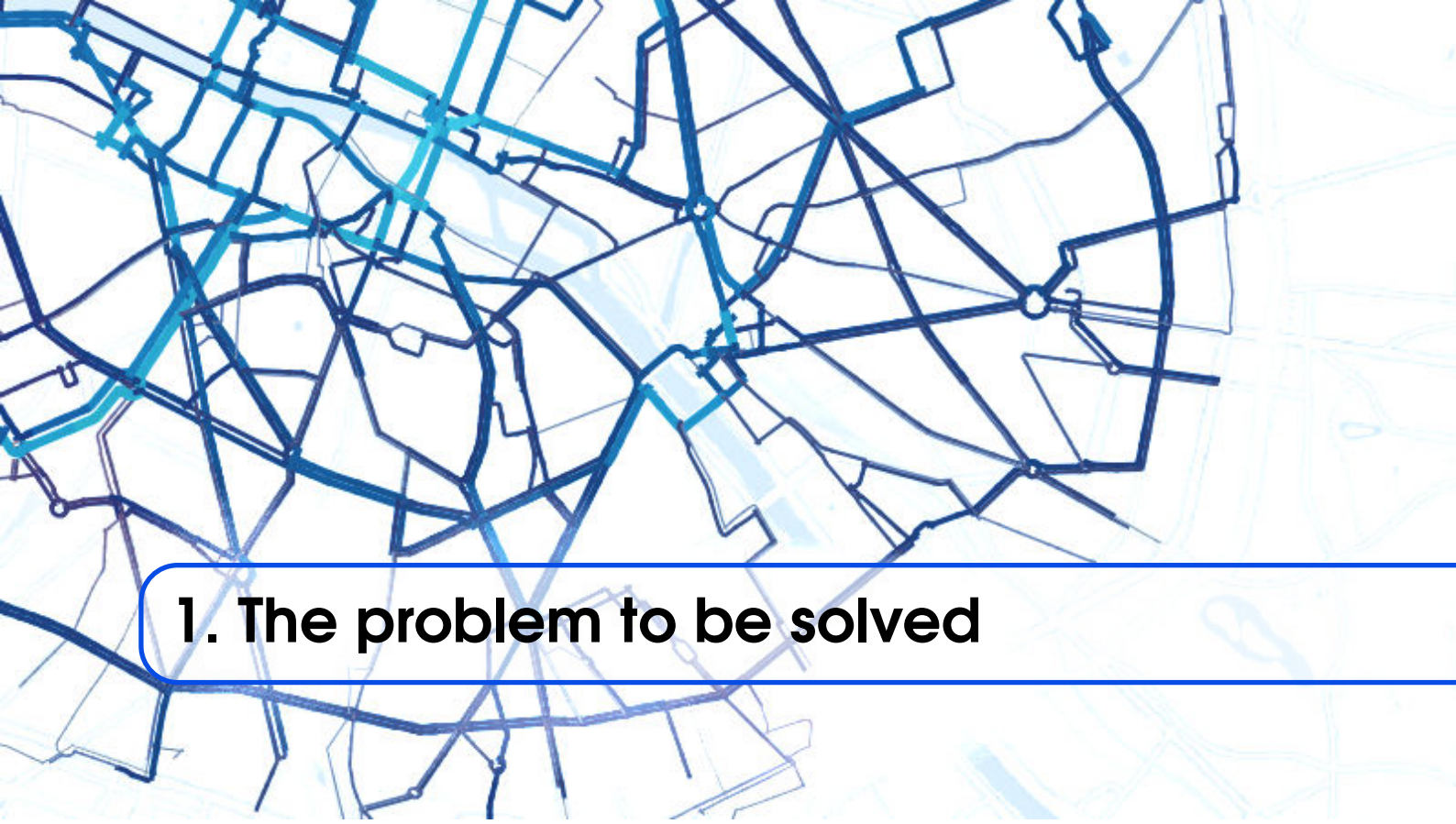
efrei

Operations Research project

Mathematics Department Efrei Paris

Year 2024/2025 S6

II



1. The problem to be solved

1.1 Introduction

The question of transport problems, like traffic, is closely linked to social, economic and ecological issues. Through the algorithms seen in class, we seek to understand how best to use a network. The nature of the costs we maximize or minimize can be human, monetary or environmental.

1.2 The projet

This project involves writing a program that solves a problem called **flow problem**. First, we invite you to work out how it works by watching **the two videos** on Moodle and then reading **a course provided in Appendix 2**. Then you'll need to build the code that solves these problems. We'll then ask you to test it on the problems in Appendix 1, providing us with execution traces. Finally, you'll need to use it to analyze the complexity generated.



2. The code

2.1 The work required

Once you've watched the videos and read the course appendix, we'll ask you to do the following task. For any flow network (V, E, c, s, t) , we ask you to code the two algorithms for finding a maximal flow : the Ford-Fulkerson method and the push-Relabel algorithm. And for any weighted flow network (V, E, c, d, s, t) , we ask you to code the algorithm for finding a minimum-cost flow for a given input flow value.

For this project, you will work with the programming language of your choice : C, C++, Python, Java.

2.2 Flow network table (V, E, c, s, t) and (V, E, c, d, s, t)

For each flow problem, $n = |V|$ is the number of vertices including s and t . The vertices will be called v_i with v_1 being s and v_n being t . The capacity of the edge between vertices v_i and v_j is always denoted $c(i, j)$. And $d(i, j)$ is the cost of the edge between vertices v_i and v_j . First of all, we'll need to create a .txt file for any given flow network, organized as follows :

n				
$c_{1,1}$	$c_{1,2}$	...	$c_{1,n}$	
$c_{2,1}$	$c_{2,2}$	...	$c_{2,n}$	
$\vdots$			$\vdots$	
$c_{n,1}$	$c_{n,2}$	...	$c_{n,n}$	

n				
$c_{1,1}$	$c_{1,2}$	...	$c_{1,n}$	
$c_{2,1}$	$c_{2,2}$	...	$c_{2,n}$	
$\vdots$			$\vdots$	
$c_{n,1}$	$c_{n,2}$	...	$c_{n,n}$	
$d_{1,1}$	$d_{1,2}$	...	$d_{1,n}$	
$d_{2,1}$	$d_{2,2}$	...	$d_{2,n}$	
$\vdots$			$\vdots$	
$d_{n,1}$	$d_{n,2}$	...	$d_{n,n}$	

FIGURE 2.1 – The tables requested in .txt.

Max-cost flow problems on the left, min-cost flow problems on the right.

In the appendices, you'll find the graphs of the ten flow problems you're asked to solve with your program. You'll need to edit the ten graphs in 10 different .txt files in the same way. These files should be attached to your rendering.

2.3 The code

2.3.1 The functions

In the program you will write, you will have to implement the following functions :

1. Read data from text file (.txt) and store in memory.

2. Display the following tables :

★ capacity matrix

★ cost matrix for flow networks with costs

★ table from Bellman's algorithm for flow networks with costs

Warning : the display function must be absolutely meticulous. Any table with columns that shift will be severely penalized. Table legibility is fundamental.

3. Algorithm for solving the max flow.

★ For F-F : details of the Breadth-first search, the potentially improving chain found with calculation of its flow value, and modifications to the residual graph.

★ For "push-relabel" : details of iterations.

4. Algorithm for solving the minimum-cost flow.

★ Bellman's detailed table.

- ★ Flow value of a potentially found improving chain.
- ★ Modifications to the residual graph.

Warning : Each function must be highlighted and clearly explained orally, using pseudo-code.

2.4 The global structure

The global structure of your program is illustrated by the following pseudo-code :

```
Start
    As long as the user decides to test a flow problem, do :
        ★ Choose the number of the problem to be processed
        ★ Read the table from a file and store it in memory
        ★ Create corresponding matrices representing this table and display
            ★ If this is a max-flow problem, choose the algorithm.
            ★ If this is a min-cost flow problem, choose the flow value
        ★ Run the corresponding algorithm
        ★ Display max flow or min-cost flow as appropriate.
End
```

2.5 Execution traces

Execution traces for the ten graphs provided in the appendices are requested in the rendering. An execution trace is what is displayed by the console. They cannot be replaced by screenshots.

For max flow problems, you will need to run your program with the two algorithms proposed.

Files will be stored as follows :

- ★ Group B - Team 4 - Problem 5 - Ford-Fulkerson : “B4-trace5-FF.txt”
- ★ Group D - Team 2 - Problem 5 - Push-Relabel : “D2-trace5-PR.txt”
- ★ Group E - Team 3 - Problem 6 - min-cost flow : “E3-trace6-MIN.txt”

Example of a trace for proposition 1 and FF :

★ Capacity table display : .

8								
0	9	5	7	0	0	0	0	
0	0	6	0	6	0	0	0	
0	0	0	0	0	7	0	0	
0	0	0	0	0	2	8	0	
0	0	3	0	0	0	0	8	
0	0	0	0	0	0	0	10	
0	0	0	0	0	3	0	5	
0	0	0	0	0	0	0	0	

The initial residual graph is the starting graph.

★ Iteration 1 :

Breadth-first search :

s

abc; $\Pi(a) = s$; $\Pi(b) = s$; $\Pi(c) = s$

bcd; $\Pi(d) = a$

cde; $\Pi(e) = b$

def; $\Pi(f) = c$

eft; $\Pi(t) = d$

Detection of an improving chain : *sadt* with a flow 6.

Modifications to the residual graph :

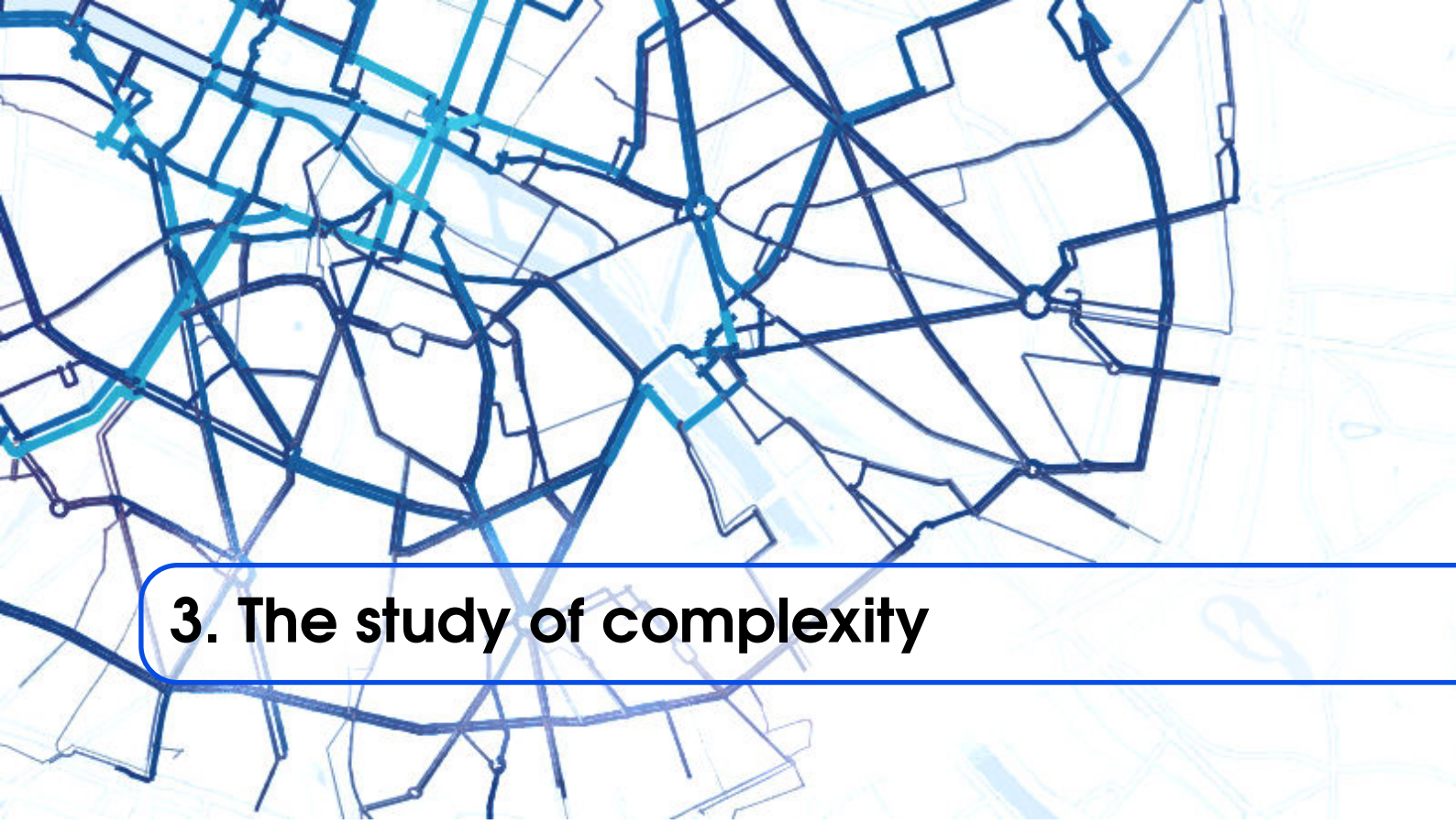
	s	a	b	c	d	e	f	t
s	0	3	5	7	0	0	0	0
a	6	0	6	0	0	0	0	0
b	0	0	0	0	0	7	0	0
c	0	0	0	0	0	2	8	0
d	0	6	3	0	0	0	0	2
e	0	0	0	0	0	0	0	10
f	0	0	0	0	0	3	0	5
t	0	0	0	0	6	0	0	0

★ Iterations 1, 2, ... :

★ Max flow display :

	s	a	b	c	d	e	f	t
s	0	8/9	5/5	7/7	0	0	0	0
a	0	0	2/6	0	6/6	0	0	0
b	0	0	0	0	0	7/7	0	0
c	0	0	0	0	0	2/2	5/8	0
d	0	0	0/3	0	0	0	0	6/8
e	0	0	0	0	0	0	0	9/10
f	0	0	0	0	0	0/3	0	5/5
t	0	0	0	0	0	0	0	0

Value of the max flow max = 20



3. The study of complexity

3.1 Introduction

This part should be considered by all teams, as long as the functions work. We now propose to study the *complexity* of the algorithms in this project.

Throughout the course of this project, we talk about complexity. But what exactly is the complexity of an algorithm? It's the evaluation of the resources required to run an algorithm (essentially the amount of memory required) and the computation time to be expected. These two notions depend on numerous hardware parameters that lie outside the realm of algorithmics : we cannot assign an absolute value to either the amount of memory required or the execution time of a given algorithm. On the other hand, it is often possible to evaluate the *order of magnitude* of these two quantities, so as to identify the most efficient algorithm within a set of algorithms solving the same problem.

This is what we propose to do here, by comparing coded algorithms.

3.2 A bit of theory

Most algorithms have an execution time that depends not only on the size of the input data, but also on the data itself. In this case, there are several types of complexity :

Définition 3.1 (worst-case complexity)

Worst-case complexity is a majorant of the possible execution time for all possible inputs of the same size. It is generally expressed in O notation.

Définition 3.2 (best-case complexity)

The **best-case complexity** is a minority of the possible execution time for all possible inputs of the same size. It is usually expressed in Ω notation.

However, this notion is rarely used, as it is often irrelevant to worst-case and average complexities.

Définition 3.3 (average complexity)

The **complexity on average** is an evaluation of the average execution time for all possible inputs of the same size, assumed to be equiprobable.

Définition 3.4 (spatial complexity)

The **spatial complexity** evaluates memory space consumption. The principle is the same, except that here we're trying to evaluate the order of magnitude of the memory volume used : it's not a question of evaluating precisely how many bytes are consumed by an algorithm, but of specifying its rate of growth as a function of the n size of the input.

3.3 The study

In this project, we'll be analyzing **worst-case complexity**. To do this, we'll ask you to generate random flow problems. Then look at the execution times of the algorithms.

3.3.1 Input flow problems

To simplify the problem, you will work with flow problems of size n vertices. The capacity matrix is always denoted $C = (c_{i,j})_{(i,j) \in \llbracket 1;n \rrbracket^2}$, as is the cost matrix $D = (d_{i,j})_{(i,j) \in \llbracket 1;n \rrbracket^2}$.

In order to generate a sampling mimicking all possible inputs of the same size n , you will write a function to edit random flow problems. This will be done as follows :

1. Take the value 0 for all $c_{i,j}$.
2. Then, for $E(\frac{n^2}{2})$ couples (i, j) with $i \neq j$, we generate an integer random number between 1 and 100 inclusive which we give to $c_{i,j}$, the E function being the integer part function.

In this way, matrix C will be zero on the diagonal and will have less than half non-zero values. For matrix D , we'll proceed in the same way. Of course, C and D have no reason to be equal, and of course, we'll give a cost to edges with non-zero capacity.

Furthermore, in the case of the min-cost flow algorithm, the user will not be asked to set the flow value. Instead, we'll take half the value of the max flow, which we'll calculate each time.

3.3.2 Time measurement

Once the problem has been generated, i.e. once C and D have been set, we'll need to store the time value for each piece of code we're interested in. In Python, for example, we simply use the function `time.clock()`, which returns the CPU time in seconds, and store this value. The difference between 2 values will give the execution time of the framed portion of code.

With this problem of size n generated, you'll need to measure the execution time of :

1. the Ford-Fulkerson algorithm. We'll call this time $\theta_{FF}(n)$,
2. the push-Relabel algorithm. We'll call this time $\theta_{PR}(n)$,
3. the min-cost flow algorithm. We'll call this time $\theta_{MIN}(n)$,

3.3.3 Point cloud

For each value of n , you will run your program 100 times with different random values for the flow problem. So, for a fixed n , you'll obtain 100 values of $\theta_{FF}(n)$, for example.

n values to be tested	10	20	40	10^2	$4 \cdot 10^2$	10^3	$4 \cdot 10^3$
-------------------------	----	----	----	--------	----------------	--------	----------------

Warning : to perform this task, you'll be working on a single processor machine. Instructions will be executed one after the other, without simultaneous operations. So don't use your machine for anything else during execution.

Once the values have been stored, you will plot, as a function of n , the following point clouds (the 100 values for the same abscissa) :

★ $\theta_{FF}(n)$.

- ★ $\theta_{PR}(n)$.
- ★ $\theta_{MIN}(n)$.

3.3.4 Worst-case complexity per algorithm

Assume that the worst-case complexity is the upper envelope of the scatterplot. For each value of n , determine this maximum value across the 100 realizations for a fixed n . Then plot the maximum value as a function of n .

For θ_{FF} , θ_{PR} and θ_{MIN} identify the type of worst-case complexity using the table 3.1 and compare your result with the course given in the project :

$O(\log(n))$	logarithmic
$O(n)$	linear
$O(n \log(n))$	quasi-linear
$O(n^2)$	quadratic
$O(n^k)$ ($k > 2$)	polynomial
$O(k^n)$ ($k > 1$)	exponential

FIGURE 3.1 – Usual qualifiers for complexity.

3.3.5 Worst-case complexity comparison

Let's now compare the two algorithms solving the same problem for n fixed by plotting :

$$\frac{\theta_{FF}}{\theta_{PR}}(n)$$

Then plot the maximum value found for each value of n and discuss the results.



4. Rendering details

4.1 The rendering

Your project must be submitted on Moodle by Saturday, May 3 at 11 :59 pm. No further delays will be accepted. On Sunday, your teachers will study your project, and the first defense will take place on Monday.

You'll need to include all your programs, ten execution traces and ten .txt files of the flow problems to be solved. Grading will take into account the quality of the algorithms and traces submitted. All teams are required to submit a complexity report of no more than 4 pages.

The report will be titled as follows : for group B and team 4 : "B4".

4.2 The oral

For the oral, a slide presentation is expected. Pedagogy and clarity are the goal of this 10-min presentation. We won't accept any code on the slides : we only ask for pseudo-code. Slides must not contain any handwritten or photographed text. All formulas and matrices must be typed.

For oral presentations, we ask you to summarize the most important points of each section. A slide summarizing the work you've done, i.e. the functions you've succeeded in making work and those you haven't, is expected. Warning : if you exceed the 10-minute presentation time, your teacher will stop you.

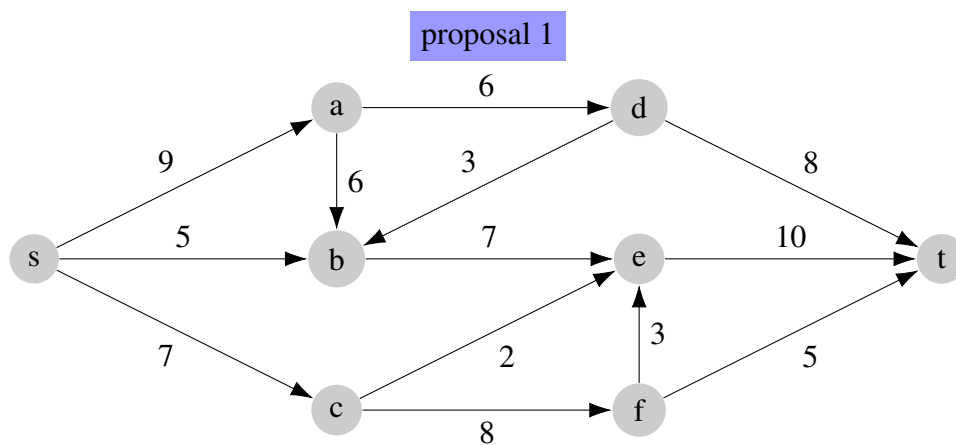
At the end of the oral, questions will be asked for 20 minutes : each student will be questioned individually on the project or on a point from the course. The code must be mastered by all team

members.

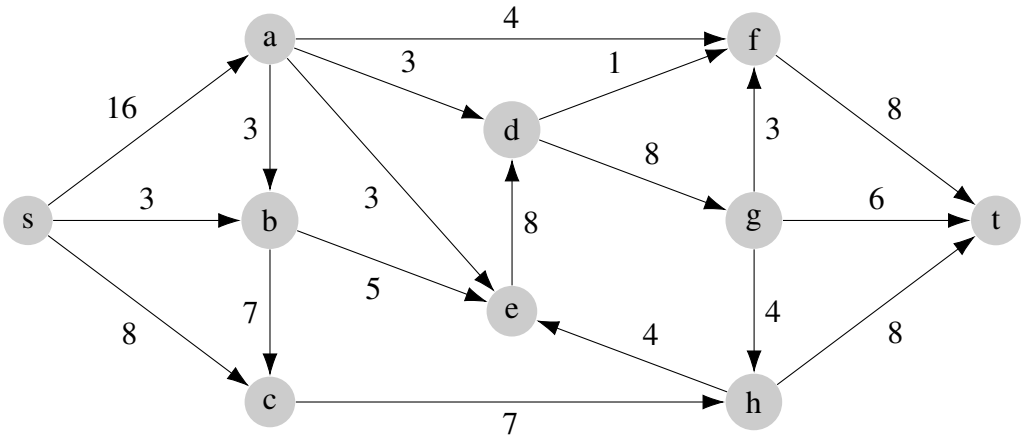
Good luck with this project,

The Operational Research teaching team.

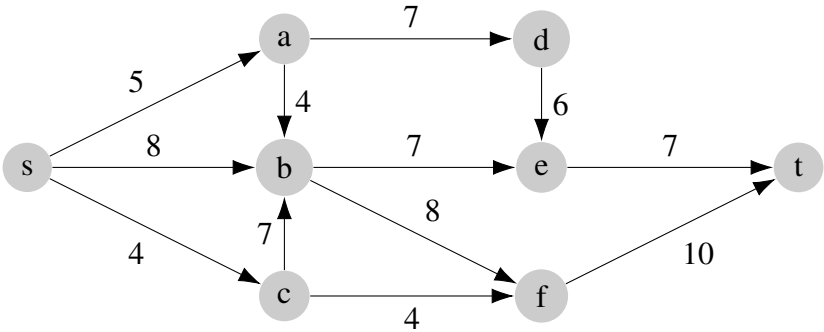
APPENDIX 1 : Ten flow proposals



proposal 2



proposal 3



proposal 4

